

A Report On

AUTONOMOUS ROVER NAVIGATION SYSTEM

Submitted By

BHAVYA VEERAMREDDY

1. INTRODUCTION

Autonomous navigation enables the rover to find and move to the target without any human intervention. In this report, I aim to design and explain an approach through which a rover could autonomously navigate toward a destination using camera input as its primary perception source.

2. REQUIRED COMPONENTS

Components	Quantity
DC Geared Motor	x4
Wheels	x4
Chassis	x1
Motor Driver (L298N)	x1
Raspberry Pi 5	x1
Raspberry Pi Camera Module	x1
LiDAR Sensor	x1
Battery	x1

3. SOFTWARE AND COMPONENT ROLES:

3.1 Camera Module

The camera provides the primary perception input for the rover. It continuously captures images of the surrounding environment, which are processed to identify the specified target and potential obstacles.

Suggested: Raspberry Pi Camera Module

Cost-Effective Alternative: 5MP OV5647 Raspberry Pi Cam Module

3.2 Raspberry Pi 5

The Raspberry Pi 5 acts as the main processing unit of the rover. It receives images from the camera, performs image processing using OpenCV, runs the YOLO object-detection model, processes LiDAR distance measurements, and uses the resulting information to make navigation and movement decisions.

3.3 LiDAR Sensor

LiDAR acts as a supplementary distance sensor. It provides distance measurements to nearby objects, allowing the rover to determine whether an obstacle is within a safe range and to obtain a more reliable estimate of the distance to the target during the final approach.

Suggested: TF-Luna LiDAR

Cost-Effective Alternative: Ultrasonic Sensor HC-SR04

3.4 L298N Motor Driver

The L298N motor driver receives the motor-control signals generated by the Raspberry Pi and supplies the required current to the DC geared motors. The motor speeds can be varied independently to control the rover's direction and movement.

4. SYSTEM INTEGRATION AND DATA FLOW:

1. Raspberry Pi Camera Module captures images of the rover's surroundings.
2. This information is then sent to Raspberry Pi through CSI Interface.
3. Raspberry Pi processes these images using OpenCV and YOLO.
4. This will help detect the target and obstacles (if any).
5. Based on camera input, Raspberry Pi decides in which direction the rover turns.
6. It converts these movement commands into control signals and PWM signals.
7. L298N Receives the converted signals.
8. L298N then adjusts the 4 DC Geared Motor speeds respectively, two on each side controlled together.

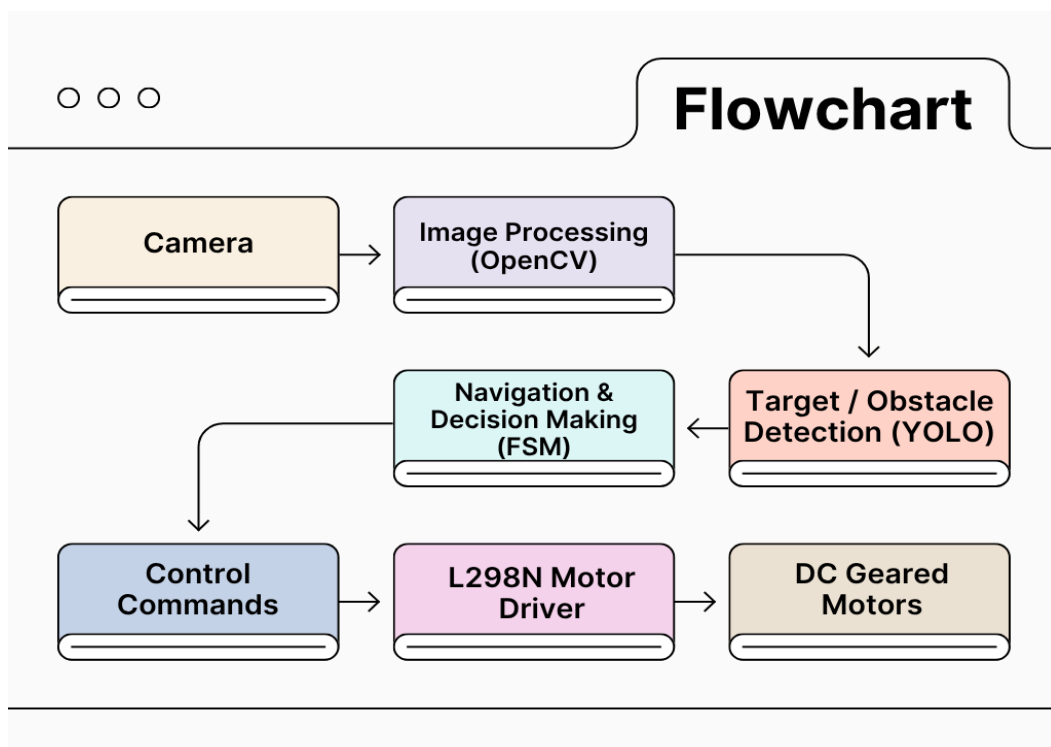


FIG 4.1 System Integration Flowchart

5. AUTONOMOUS NAVIGATION FLOW

1. Enter target class.
2. Rover rotates 360°.
3. Rover finds the target.
4. Rover moves to the target.
5. Stop once the rover reaches the target.

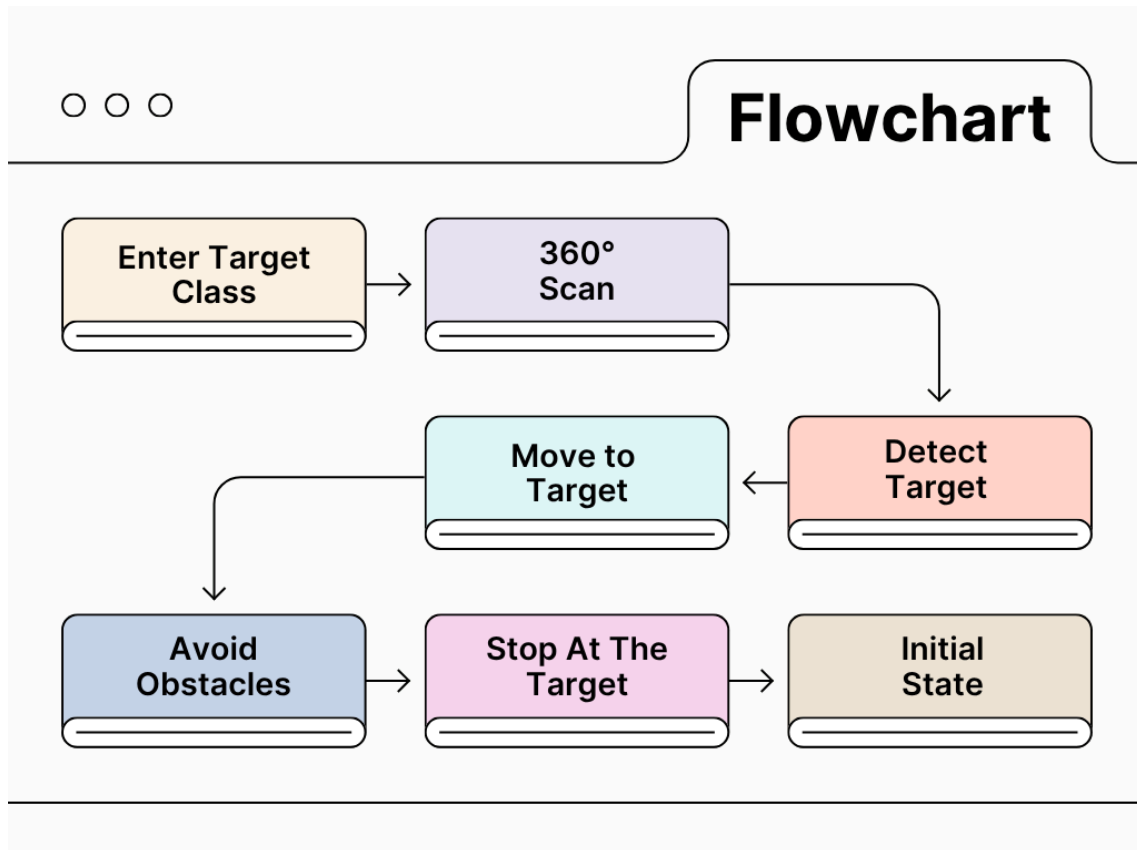


FIG 5.1 Autonomous Navigation Flowchart

5.1 Entering Target Class

The user specifies the object that the rover must locate. Two cases are considered based on the type of information provided by the user

CASE 1: TARGET IMAGE

A reference image of the target is uploaded. The image is processed using YOLO to identify the corresponding object class.

Consideration: The target image may not represent the target accurately under different viewing angles and lighting conditions.

CASE 2: TARGET CLASS

The user specifies the target object class directly, such as “bottle”, ”chair”, ”person.” The rover uses this input as the target class for object detection and searches its surroundings for an object belonging to the specified class.

- The specified class must be present in the YOLO model’s trained classes.
- For a custom target, the pre-trained YOLO must be fine-tuned/trained to recognise the class.

Selected Case: Target Class Input

★ *The system uses target-class input to avoid complexity*

5.2 Step-2: 360° Scan

The rover rotates through 360° while processing camera frames with OpenCV and YOLO to locate the specified target.

5.2.1 Factors Affecting Target Detection During Rotation

1. Rotation speed of the rover
2. Size of target
3. YOLO processing speed

$$\Delta\theta = \frac{\omega}{f_{YOLO}}$$

Where ω = Speed at which rover rotates in degree/sec

f_{YOLO} = Processing in FPS

$\Delta\theta$ = Degree Rotation of Rover

This formula gives the angle through which the rover rotates between two YOLO-processed frames. At $f_{YOLO} = 5$ FPS and $\omega = 5^\circ/\text{sec}$, the rover rotates approximately 1° between consecutive processed frames.

The rotation speed of the rover should be selected according to the YOLO processing speed, such that minimum angle gap lapses between consecutive processed frames, reducing the possibility of missing the target.

It will take the rover around 72 seconds for 360° rotation. The rotation speed can be adjusted based on how wide the target object is, and could be rotated faster if the object is wider as YOLO doesn't need the entire target to detect.

Additional Feature For Increased Accuracy And Precision:

- ★ Target can be declared as detected in 2-3 consecutive frames before being declared as FOUND.

5.2.2 Optional Target Colour Verification

An additional feature could be added where we enter colour details of the target. OpenCV can be used to analyse and determine the colour of the target, which can help us pinpoint our target in a situation where multiple objects belonging to the same object class are present.

5.2.3 Target Not Detected

If the target is not detected during the initial 360° search, the rover performs an “obstacle-aware exploration” by moving to a new viewpoint and repeating the search.

5.3 Move to Target

Keep the detected target at the centre of the camera frame while moving towards it.

5.3.1 Target Position

- If the target is to the left of the frame centre —> Rover turns left.
- If the target is to the right of the frame centre —> Rover turns right.
- If the target is near the frame centre —> Rover moves forward.

5.3.2 Determining the Target Position

- Assign an x-axis to the camera frame.
- Determine the centre x-coordinate of the frame. This is the frame centre.
- Obtain the centre x-coordinate of the YOLO bounding box.

- $x_{target} < x_{frame} \rightarrow \text{left}$
- $x_{target} > x_{frame} \rightarrow \text{right}$
- $x_{target} \approx x_{frame} \rightarrow \text{centred}$

5.3.3 Motor Control

- Turn left —> Left motors slow down and right motors speed up.
- Turn right —> Right motors slow down and left motors speed up.
- Move forward —> Both side motors move forward at similar speeds.

5.4 Obstacle Avoidance

5.4.1 Camera-Based Obstacle Detection

The camera frame is divided into three regions: left, centre and right. When an obstacle is detected, a bounding box can be used to determine its position within the frame. The available space on either side of the detected obstacle is compared. The rover turns toward the side with greater available space. To account for the rover's physical width and additional clearance requirements, a safety margin is considered when determining whether sufficient space is available for the rover to pass.

$$M = \frac{W_R}{2} + C$$

M = required lateral safety margin

W_r = rover width

C = additional clearance

Consideration:

★ *The bounding box is in pixels, so the physical margin has to be converted into the appropriate image-space margin which adds to the degree of complexity.*

5.4.2 LiDAR-Assisted Obstacle Avoidance

The LiDAR readings can be used to determine whether the detected obstacle is within a predefined safety distance. If an obstacle is too close, the rover compares the available space on its left and right sides and turns toward the safer direction. This provides more reliable physical clearance than relying solely on the obstacle's position in the camera image.

★ *The camera tells the rover what and where the obstacle is, LiDAR tells the rover how far away it is.*

5.5 Stop At Target

Once the rover approaches the detected target, LiDAR is used to measure the distance between the rover and the target. When the measured distance falls below the stopping threshold, the rover stops the motors.

★ *The camera remains responsible for target detection and positioning, while LiDAR provides supplementary distance information for accurate stopping.*

6. ADDITIONAL FEATURES

A 2D scanning LiDAR can be used to create a local map of obstacles. It could be incorporated between the sensing layer and the navigation & decision making layer. The rover will remember the previous locations of the obstacles, instead of just relying on the present camera frame. This will improve path planning and autonomy of the rover.

7. INCLUDED FEATURES

- LiDAR-Assisted Obstacle Avoidance
- LiDAR-Based Target Stopping
- Target Colour Verification
- Target-Not-Found Exploration

8. REFERENCES

[1] Y. Cheng, K. Hayden, Z. Aliyazicioglu, and T. Lin, “*Autonomous Navigation System Design for a Smart Robotic Rover*,” California State Polytechnic University, Pomona.

[2] T. Tariq, N. M. Ansari, R. Iqbal, S. Rana, S. Salman, R. Ali, and S. W. Alam, “Design and Implementation of Autonomous Rover Using Computer Vision Algorithm,” *Jilin Daxue Xuebao (Gongxueban) / Journal of Jilin University (Engineering and Technology Edition)*, vol. 42, no. 01, pp. 112–117, Jan. 2023, doi: 10.17605/OSF.IO/S3AGF.

9. RELEVANT DEMONSTRATION VIDEOS

[1] <https://youtu.be/3uKE2qJNbyE>

[2] <https://youtu.be/3uKE2qJNbyE?t=131>